

[41052 Advanced Algorithms - Assignment 1]

Track B – Implementation and Building Something

Consistent Hashing

Cache Router Visualiser & Analysis

Truong Giang Nguyen – 25520038

Contents

1	Executive Summary	2
2	Consistent Hashing Algorithm	3
2.1	The Problem of Simple Hashing Algorithm	3
2.2	Consistent Hashing Algorithm Advantages	4
2.3	Design Decisions	4
2.4	Differences from Textbook Implementation	5
2.4.1	Simplifications	5
2.4.2	Extensions	5
3	The Tool: Cache Router Visualiser	5
3.1	Frontend & Example	5
3.2	Cache Router Architecture	7
3.3	Interface Decisions	8
4	What I Learned	9
4.1	Design Patterns	9
4.1.1	Strategy Pattern	9
4.1.2	Factory Pattern	9
4.2	Realistic Traffic Simulation	9
4.3	Sliding Window Hit Rate Calculation	10
4.4	Verifying the Result Against the Calculated Hypothesis	10
5	AI Use Disclosure	10
6	Reference	11

1 Executive Summary

I implemented consistent hashing as a hash ring with virtual nodes, in C++23, and built a Cache Router on top of it. The program is an in-memory sharded cache served over HTTP, with a browser interface that visualises where every key lands and measures what happens when the cluster changes shape. The calculation on paper shows that on a cluster of size n , consistent hashing preserves n times more keys than simple hashing when the adding or removing a node. The program's output aligns with the calculation, as for $n = 10$, a simple router remapped 91% of the keys, while a consistent router only remap around 10%.

2 Consistent Hashing Algorithm

Caching is a technique to store frequently accessed in a data store for lower latency by avoiding a database query for every request. In distributed system, instead of relying on a single cache, multiple cache nodes (a cache cluster) are used to improve the system resilience. This projects implement a toy cache router in C++, with an interactive web view frontend to visualise the differences between simple hashing consistent hashing. The user can switch the routing strategy, add/remove cache nodes and simulate realistic requests traffic consists of different keys with varying query frequency.

A cache router must distribute the requests evenly to all available cache nodes in the cluster, while routing the read requests to the correct cache node that stores the data. In order to do that, the router must answer one question on every request: which node holds this key?

2.1 The Problem of Simple Hashing Algorithm

The naive answer is to hash the key into an integer, then take the modulo with the cluster size N . This technique distributes evenly among all nodes, and it is completely deterministic. However, this approach fails when the size of the cluster is variable.

Construction

Let $h : \text{key} \rightarrow \mathbb{N}$ be a uniformly distributed hash function. A key k in a cache cluster with n nodes is mapped as follow:

$$\text{node}(k) = h(k) \bmod n$$

Since the $h(k)$ is uniformly distributed, the probability for key k to be mapped to i for $i \in [0, n - 1]$

$$P(\text{node}(k) = i) = P((h(k) \bmod n) = i) = \frac{1}{n}$$

Adding a new node requires remap for almost all keys

The probability for key k to be mapped to i after adding a new node:

$$\text{node}'(k) = h(k) \bmod (n + 1)$$

$$P(\text{node}'(k) = i) = P(h(k) \bmod (n + 1) = i) = \frac{1}{n + 1}$$

For a key to be kept in the same node, we need $h(k) \bmod n = h(k) \bmod (n + 1) = i$. Since n and $n + 1$ are always coprime, $h(k) \bmod n = i$ and $h(k) \bmod (n + 1) = i$ are independent. Therefore, $\text{node}(k)$ and $\text{node}'(k)$ are independent. Hence:

$$\begin{aligned} P(\text{stay}) &= \sum_{i=0}^{n-1} P(\text{node}(k) = i) * P(\text{node}'(k) = i) \\ &= n * \frac{1}{n} * \frac{1}{n + 1} = \frac{1}{n + 1} \end{aligned}$$

For a reasonable size cluster $n = 10$, adding a node meaning that only $\frac{1}{11}$ of the number of keys stays, resulting in a remap of roughly 90% remaining keys.

In a distributed environment, the system must scale elastically, as the number of nodes increases or decreases depending on the traffic demand. Therefore, simple hashing is not ideal for this use case.

2.2 Consistent Hashing Algorithm Advantages

Consistent hashing solves the rehashing problem by mapping both keys and nodes onto the same circular space with multiple replicas per node, rather than relying on the cluster size n directly.

Construction

Let the hash function $h : \text{key} \rightarrow [0, M)$ map both keys and node identifiers onto a ring of size M (in practice $M = 2^{64}$ using a 64-bit hash). Each node S_j is placed on the ring at position $h(s_i)$. A key k is assigned to the first node to the right side of the hashed position in the ring:

$$\text{node}(k) = \begin{cases} \arg \min_{s_i \in S} \{h(s_i) : h(s_i) \geq h(k)\} & \text{if such } s_i \text{ exists} \\ \arg \min_{s_i \in S} \{h(s_i)\} & \text{otherwise (wrap around)} \end{cases}$$

where $S = \{s_1, \dots, s_n\}$ is the set of node positions on the ring.

Each node therefore owns the arc of the ring between itself and its counter-clockwise neighbour. The n node positions partition the ring into disjoint arcs, whose lengths sum to M .

Adding a node only remaps keys in one arc

Each node s_r owns the arc between itself and its counter-clockwise neighbour s_l : $(h(s_l), h(s_r)]$.

When a new node s_{n+1} is added at position $h(s_{n+1})$, it lands inside exactly one existing arc. Without loss of generality, assume $h(s_{n+1})$ lands in the arc owned by node s_r , splitting it into two sub-arcs:

$$(h(s_l), h(s_{n+1})] \quad \text{and} \quad (h(s_{n+1}), h(s_r)]$$

- Keys with $h(k) \leq h(s_{n+1})$ are reassigned to s_{n+1} .
- Keys with $h(k) > h(s_{n+1})$ keep their original assignment to s_j .
- Keys in every other arc are untouched, since $h(s_{n+1})$ does not fall inside them.

Only one arc is splitted, so at most one node's worth of keys is remapped. Hence, the probability of a key getting remapped is equal to the probability of the key living in the splitted arc.

Since h uniformly distribute k and $n + 1$ nodes around the ring, each node s_i accounts for an arc with $\frac{1}{n+1}$ the circumference of the whole ring, therefore:

$$P(\text{stay}) = 1 - P(\text{remap}) = 1 - \frac{1}{n+1} = \frac{n}{n+1}$$

Comparing to Simple Hashing, this is much more efficient by maintaining n times more nodes. For $n = 10$, adding a node only requires roughly 9% of the nodes to be remapped.

Removing a node reallocate the its owned arc to the neighbour node. This can put extra stress on one node and lead to uneven load distribution. To prevent this, multiple virtual nodes (replicas) are placed on the ring and all point to one real node. When the node is removed, the scattered positions on the ring equally assigned the removed space to multiple nodes, still preserving the even distribution.

2.3 Design Decisions

The project follows C++23 standards, and is built using CMake 4.0. The project consists of 2 major components: the core cache router library, and the HTTP server to expose API endpoints to the frontend. External libraries include `cpp-httpplib`, `json`, `GoogleTest`, and `smhasher` for the MurmurHash3 function.

The project followed OOP principles and implemented several design patterns such as the Strategy and Factory pattern to decouple the routing and evicting logic from the actual container interface, allowing more flexible options when interacting with the frontend. Modern C++ idioms such as `std::optional`, `std::variant` and `std::format` are also utilised wherever practical, both as a good practice and as a deliberate learning goal for this project.

2.4 Differences from Textbook Implementation

The consistent hashing implementation is proposed by proposed by Karger et al. (1997). This project simplify the mathematical factor of the implementation, allowing a more flexible user customisations at a cost of losing the proposed optimal configuration.

2.4.1 Simplifications

According to the implementation the hash ring allocate a key to the nearest node on the hash ring, while this projects uses `std::map::lower_bound` to route a key to the counter-clockwise neighbour. As a result, the complexity of querying a key matches with the proposed implementation at $O(\log(nv))$ (v represents the number of virtual nodes). Adding nodes is $O(v)$, however, removing nodes costs $O(nv)$ due to a linear search in the map.

2.4.2 Extensions

Routing algorithm can be swapped at runtime, with the existing topology replayed into the new one. Eviction policies include Least Recently Used (LRU) and Least Frequently Used (LFU) are constructed with the cache nodes, with a customisable size and per-node virtual nodes count. The textbook implementation uses a fixed number of points for each node, which grows logarithmically to the number of cache nodes (Karger et al., 1997).

3 The Tool: Cache Router Visualiser

3.1 Frontend & Example

The cache router frontend serves a single page that is a live view of a running cluster. The user can add and remove nodes with a customisable capacity, policy and replica count; switch the routing strategy; and clear data, clear nodes or restore the default ten-node cluster. A built-in traffic generator simulates realistic request load so the cluster can be watched under pressure.

Instead of abstracting away the routing logic in the backend, this visualiser tool allows user to view the hash ring in practice, with real-time update along the traffic simulation. The user can experiment with different scenarios, for example, removing a node in the middle of the simulation. Statistics such as hit rate, cache node capacity, as well as the number of invalidated keys when the cluster size change are live updated.

The web frontend is served by a HTTP server defined in `src/server/main.cpp`, which is the entry point of the program. The program first construct a `CacheCluster` with the default configuration, then mount the static web directory and register the API endpoints defined in `api.hpp`. The detail API endpoints specification will be covered in Section 3.3.

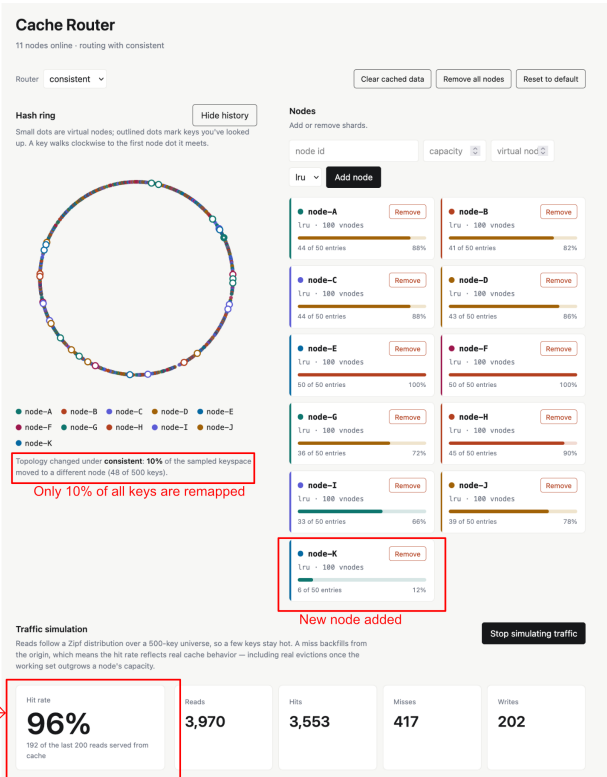
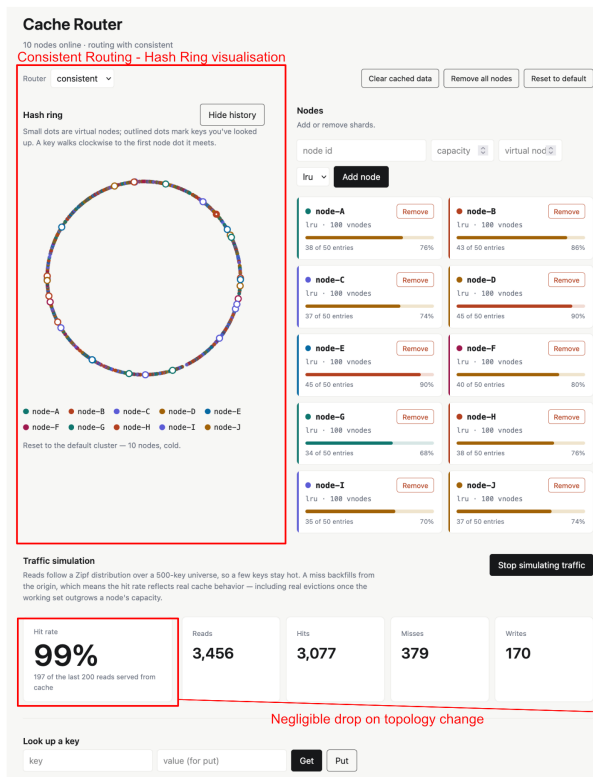
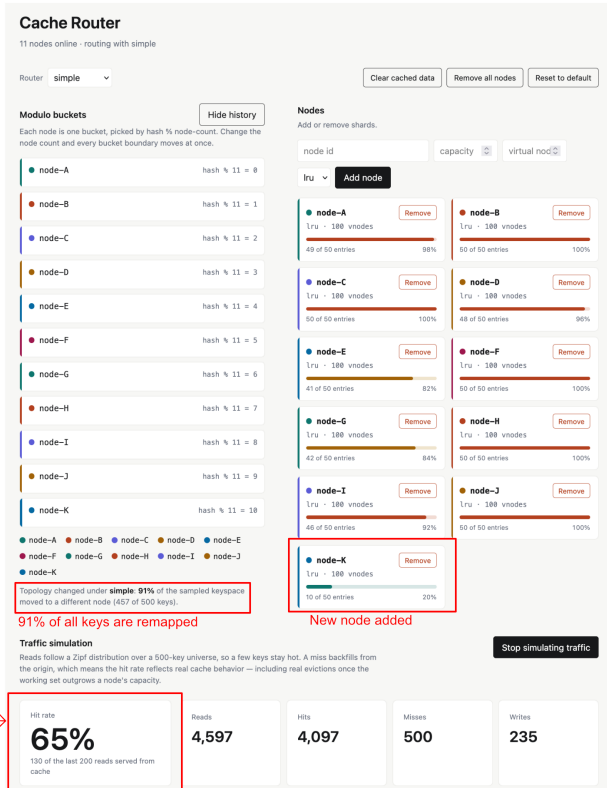
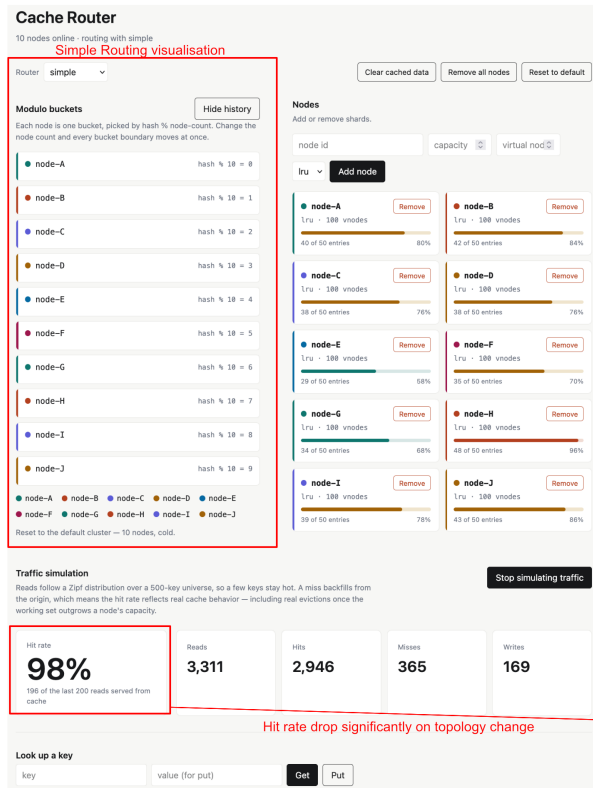


Figure 1: Comparing Simple Hashing and Consistent Hashing using the Cache Router Visualiser

The 2 examples compare Simple Hashing and Consistent Hashing in the same topology changes. Initially, there are 10 cache nodes, all with a capacity of 50 and 100 virtual nodes. Both examples are executed after 60 seconds of traffic simulation to populate the cache with some initial data. Another node is then added to cluster, simulating a scaling operation in distributed system.

The first example shows the high impact of a topology change to a Simple Hashing router, where 91% of the keys have to be remapped. This number is consistent with the previous calculation. As a result, the hit rate temporarily dropped to 65%. This operation can be very costly in a production environment, as each cache miss requires a database query, which increase the latency and put more stress on the database server.

On the other hand, Consistent Hashing shows a very subtle impact. As calculated before, only around 10% of the keys are remapped, causing a slight drop of 3% to the hit rate. This comparison perfectly highlight the advantages of consistent hashing, explaining why it is widely applied in distributed system.

3.2 Cache Router Architecture

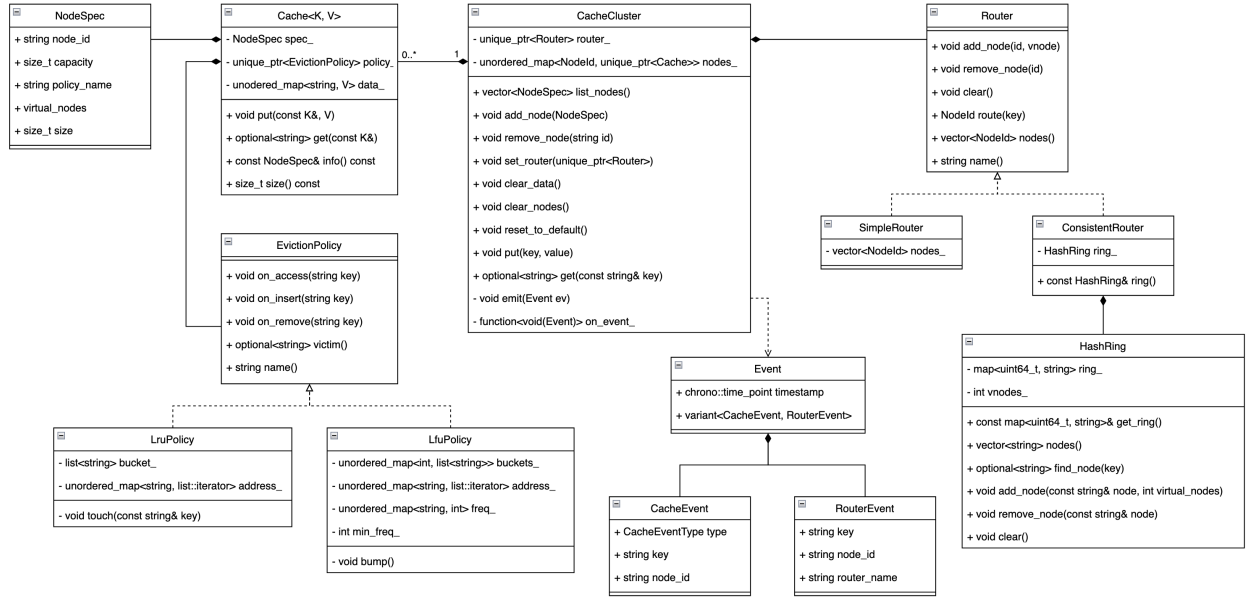


Figure 2: Cache Router backend Class Diagram

The backend is a standalone C++ library (`cachrouter::core`). The web server only calls into the library based on the API calls, so the routing and caching logic can be tested without any network calls.

- **HashRing** implements the consistent hashing algorithm. The ring is a `std::map<uint64_t, std::string>` from ring position to node ID. Each node is inserted at its own hash and at one more position per virtual node, hashed from the string "`<node>:<i></i>`". The node is located using `std::lower_bound` on the hashed key, and wraps around to `begin()` when it reaches the end.
- **Router** routes the request to the correct node. `ConsistentRouter` wraps a `HashRing`, while `SimpleRouter` is the baseline $h(k) \bmod n$ over a vector of node IDs.
- **Cache and EvictionPolicy**. The cache does not decide which key to evict but queries `EvictionPolicy` to find the victim entry based on `LruPolicy` or `LfuPolicy`.

- **CacheCluster** is the single entry point of the library. It owns one `Router` and all the `Cache` nodes. For a `put` or `get`, it asks the router for the node ID, forwards the call to that `Cache`, and emits an `Event`. For topology changes, it forwards the call to `Router` to add/remove nodes.

An `Event` is a timestamp and a `std::variant<CacheEvent, RouterEvent>`. A `RouterEvent` records which router sent which key to which node, and a `CacheEvent` records what happened on that node (e.g. a hit or a miss). `CacheCluster` passes each event to a `std::function` callback, which is sent to a logger. This architecture keeps the library separated from the server, and can be independently tested.

3.3 Interface Decisions

The frontend communicates to the backend via RESTful APIs. The endpoints are defined as follow:

- `GET /api/nodes` : Querying all available nodes
- `POST /api/nodes` : Adding a new node. The node specification is attached in the request body.
- `DELETE /api/nodes/:id` : Removing an existing node. Non-existing node returns a 404 Error.
- `PUT /api/cache/:key` : Returns 201 Created and the hashed key for visualisation.
- `GET /api/cache/:key` : Returns a payload containing the key, value, and the hash. Cache miss returns only the hash, with an explicit missing error.
- `GET /api/router` : Returns the router topology (router type, available nodes, ...).
- `POST /api/router` : Switch the router type (consistent router or simple router).
- `POST /api/reset` : Reset the router topology to the default 10 nodes configuration.

4 What I Learned

4.1 Design Patterns

When building this project, I encountered many architectural concerns. This project consists of multiple components interacting with each other. To ensure the code maintainability and extensibility.

4.1.1 Strategy Pattern

The strategy pattern is used in this project. In particular, it is used to decouple the logic from the data container. Firstly, `Consistent` and `SimpleRouter` separate the routing logic from the `CacheCluster`. The cluster stores the pointers to all cache nodes in a `std::unordered_map`. The `EvictionPolicy` are also separated from the cache store, with pluggable `LruPolicy` and `LfuPolicy`. The strategies are passed into the container via dependency injection.

```
// Storing the pointer to the policy as a class member  
// Usage: policy_ -> victim();  
class Cache {  
    std::unique_ptr<EvictionPolicy> policy_;  
    ...  
};  
  
// Injecting the routing logic into the cache cluster  
void CacheCluster::set_router(std::unique_ptr<Router> router);
```

By separating the routing and caching logic, it allows user to switch the routing and caching policy independently at runtime from the frontend. It also make the program more extensible for future development, as other policies can be implemented based on the existing interface, without needing to care about data storing.

4.1.2 Factory Pattern

The frontend only sends names as strings, so the factory functions were used to turn a string name into an object. This keeps the knowledge of the concrete classes in one place, so the server and `CacheCluster` never need to include `LruPolicy` or `ConsistentRouter`.

4.2 Realistic Traffic Simulation

I initially used a uniformly random keys generation with a 50-50 read to write ratio to simulate the requests traffic. However, this simulation is not realistic, as it assumes that all data are accessed at an even frequency. As a results, most of the keys are evicted before they got accessed a second time, creating a high a mount of cache miss.

Instead, I simulated a traffic with varying frequency, i.e. some keys are more likely to be retrieved than others. These keys are kept in the cache and not evicted, and the hit rate is simulated correctly. To achieve this, I used a Zipfian distribution, making lower-value keys more accessed more frequently, and hence should not be evicted. I also create a read-heavy program to highlight the advantages of caching, by making 5% of the traffic to be write requests. A cache miss writes back to the cache, as a real cache would behave.

4.3 Sliding Window Hit Rate Calculation

At first I calculate the hit rate as cumulative hits over cumulative reads, which is the obvious definition. However, this makes the statistics less sensitive to topology changes and harder to illustrate the difference between Simple Hashing and Consistent Hashing. After a few thousand reads the changes barely create any impact, because each new sample accounts for less fraction of the total.

4.4 Verifying the Result Against the Calculated Hypothesis

When researching about consistent hashing, I wanted to know how much difference the algorithm would make in a particular use case. I wrote a simple mathematical construction of both simple hashing and consistent hashing by hand, then asked AI to suggest a reasonable sized example to plug into the formula. As it turns out, my calculation aligned with the result of the program. Being able to reason an output of a program mathematically helps me deeply understand the algorithm, and also help me spot an unexpected behaviour easier.

5 AI Use Disclosure

This projects make uses of AI tools to help in the development process. The AI tools include:

- **Claude.ai Chatbot** for asking questions and explanation relating to project architectural design and development environment setup.
- **Claude Code** for implementation and code review, especially the frontend UI.

I used Claude Chatbot to design the project structure. One valuable suggestion that it gives me is the use of the Strategy design pattern for more flexible runtime configuration. It also helped me in conducting research about the implementation of the consistent hashing algorithm and verifying my mathematical formulas.

Claude Code is used as an implentation agent. I used it extensively to develop the frontend interface. I treated it like a frontend designer, prompting it to generate code in `index.html`. As more features being developed in the frontend, I make sure that it only uses the existing API endpoints, by restricting its permission when editing the backend. Any changes to the core backend must be manually reviewed and tested.

The frontend Claude Code developed initially used a simple uniformly random traffic generation. The statistics was not matching my expectation, so I describe the problem and my assumptions to Claude chatbot. The chatbot then give me suggestions on how to resolve the issue. It suggested simulating the traffic with varying frequencies for different keys using a Zipfian distribution. I also realised that the cumulative hit rate calculation Claude Code implemented was too insensitive to changes, so I prompted it to only consider the 200 most recent requests in the calculation. I trusted Claude Code on the frontend implementation, but I still manually test the frontend usability and give it feedback for improvements.

6 Reference

Karger, D., Lehman, E., Leighton, T., Panigrahy, R., Levine, M., & Lewin, D. (1997). *Consistent hashing and random trees*. Proceedings of the Twenty-Ninth Annual ACM Symposium on Theory of Computing - STOC '97, 654–663. <https://doi.org/10.1145/258533.258660>